GraphMemShield: Auditing and Mitigating Cross-Session Privacy Leakage in Dynamic Knowledge-Graph Memories for Graph-Backed Applications

Anonymous Author(s)

***Abstract*—Modern graph-backed applications increasingly rely on persistent knowledge-graph (KG) memories that ingest entities, relations, provenance, and timestamps from one user session and reuse them in later sessions. While prior work studies leakage from model parameters, prompt context, or static graph indices, the privacy boundary of a *dynamic, multi-user, multi-session* KG memory remains under-examined. We show that even when raw text is hidden, the combination of cross-session retrieval, structural fingerprints, and write-time metadata leaks information about the underlying user. We introduce GraphMemShield, an auditing framework comprising (i) a formal model of dynamic KG memory with provenance-aware edges, (ii) three baseline attacks—CROSSSESSIONPROBE, SESSIONGRAPHLINK, and TEMPORALPATHINFER—that quantify edge exposure, session re-identification, and write-order inference, and (iii) two defenses, GRAPHMEMGUARD for provenance-filtered retrieval with bounded cross-session exposure and RANDOMIZEDEDGEADMISSION as a write-time edge-suppression proxy. We evaluate the framework on a synthetic multi-session graph and on a Docker-backed mock deployment that ingests a de-identified JSONL dialogue corpus through a MongoDB-backed cloud API. On the batched Docker workload (12 sessions, 6 users, 48 edges) the unmitigated probe exposes *all* 48 victim-owned edges across 48 adversarial queries, while strict provenance/session isolation reduces exposure to zero. A query-budget analysis demonstrates that a single entity-anchored probe per session is sufficient to surface every target edge under the baseline policy, and that allowing $b{=}1, 2$ cross-session edges per session pair tightly bounds residual exposure. We additionally report a ε-parameterised sweep of the write-time admission proxy and discuss how the observed gap between strict isolation and bounded sharing motivates principled privacy–utility analysis. We position our results as a reproducible *benchmark and defensive baseline*—rather than a claim of end-to-end differential privacy—and outline the additional adjacency, accounting, and dataset work required to lift the proxy mechanism to a fully provable guarantee.**

***Index Terms*—Knowledge-graph memory, cross-session leakage, provenance, graph-augmented retrieval, differential privacy, session linkage, auditing.**

I. INTRODUCTION

GRAPH-BACKED applications now sit at the intersection of conversational interfaces, graph-augmented retrieval, and persistent memory services. A single deployment may parse user utterances into $(\textit{subject}, \textit{relation}, \textit{object})$ triples, store them in a property graph or KG index, and later retrieve k-hop neighbourhoods to ground responses for the *same* or for a *different* user. The resulting backend is no longer a passive read-only knowledge base: it is a continuously evolving multi-tenant graph whose retrieval behaviour is shaped by every prior session.

Recent privacy research has established that text-level memory can leak verbatim facts [1], [2], that retrieval-augmented systems expose the documents in their indices [3], [4], and that learned graph models suffer from link-stealing and embedding-inversion attacks [6]–[8]. Yet the deployment pattern that combines all three—*a dynamic, multi-user KG memory consulted by a black- or gray-box retrieval service*—has received limited focused attention. In particular, no existing benchmark we are aware of jointly measures (i) cross-session edge exposure, (ii) anonymized-session linkage by graph structure, and (iii) write-order inference from observable provenance metadata, all on the *same* graph, with the *same* defensive policy applied at retrieval time.

Problem. A user u_v confides a sensitive relation—*visited a cardiology clinic*—to a graph-memory service. A different user u_a later asks an unrelated question whose 1-hop graph expansion reaches the same entity nodes. Without a provenance-aware policy, the service returns edges *owned by* u_v, and the textual response reveals or strongly hints at the sensitive fact. Even if responses are sanitised, the observable *retrieved subgraph*, its *insertion order*, and its *degree signature* can be used to: (a) re-identify u_v's sessions across pseudonymised IDs and (b) reconstruct the sequence of writes that introduced the sensitive neighbourhood.

Contributions. This paper makes the following contributions:

- We formalise dynamic, multi-session KG memory with provenance-aware edges and define three concrete privacy failure modes: cross-session retrieval contamination, session-linkage leakage, and temporal-path exposure.
- We instantiate three reproducible baseline attacks—CROSSSESSIONPROBE, SESSIONGRAPHLINK, and TEMPORALPATHINFER—and a unified *leakage event/unique edge* accounting that supports both single-shot and budgeted query regimes.
- We present GRAPHMEMGUARD, a retrieval-time defense combining provenance filtering, blocked-sensitivity sets, and a bounded per-pair exposure budget, together with RANDOMIZEDEDGEADMISSION, a seeded write-

Manuscript prepared for IEEE Transactions on Information Forensics and Security. The associated open-source artifact is available at `https://github.com/SonHaXuan/GraphMemShield`.

time admission proxy that exposes the privacy–utility surface of a future differentially private mechanism.
- We release an open-source artefact (Python package, unit tests, JSONL ingestion, Docker integration) that reproduces all figures and tables in this paper, including a 62-row aggregate benchmark across the synthetic and Docker-backed datasets.
- We document the framework's deliberate scope: the strict GRAPHMEMGUARD configuration is reported as a *provenance/session isolation baseline*, and the admission mechanism is a *seeded proxy* pending formal accounting.

Roadmap. Section II surveys prior work on memory leakage, retrieval-augmented privacy, and graph privacy. Section III defines the system and threat models. Section IV presents the GraphMemShield abstractions, attacks, and defenses. Section V describes the open-source implementation, including the Docker-backed adapter. Section VI reports synthetic and batched results. Section VII discusses scope, limitations, and ethics. Section VIII concludes.

II. RELATED WORK

Memory and prompt leakage. Carlini *et al.* [1] demonstrated that large language models memorise and emit training data verbatim under crafted prompts. Subsequent work expanded this to fine-tuned conversational systems [2]. These attacks treat the model itself as the leaking surface. Our work assumes that the model is benign and focuses on the *external memory store* that mediates retrieval across sessions.

Retrieval-augmented generation (RAG) privacy. Zeng *et al.* [3] formalised leakage from RAG document indices, and Qi *et al.* [4] demonstrated chunk extraction by adaptive querying. Graph-augmented RAG has its own extraction surface; recent work [5] extracted entities and relations from graph-backed RAG by exploiting the k-hop expansion. GraphMemShield differs by treating the KG as a *multi-user, multi-session, append-only* structure and by isolating the cross-session boundary as the primary privacy artefact.

Graph privacy. Link-stealing [6], graph model inversion [7], and embedding-leakage attacks [9] target trained GNNs and learned representations. Differentially private graph release mechanisms [10]–[12] and DP graph neural networks [13], [14] provide formal guarantees on *static* graphs or *model parameters*, not on the live retrieval pipeline of an evolving multi-user memory. Our framework is intentionally aligned with these formalisms (we adopt edge-event adjacency) so that future versions of RANDOMIZEDEDGEADMISSION can plug in established DP machinery.

Membership inference and re-identification. Membership inference attacks [15], [16] reveal whether a record participated in training. Graph-level re-identification [17] has shown that behavioural traces such as walks or interaction histories serve as fingerprints. SESSIONGRAPHLINK adapts this insight to *retrieval-time graph fingerprints*, ranking anonymised sessions by structural similarity rather than by raw entity overlap.

Provenance-aware access control. Provenance graphs are widely used in audit and compliance pipelines [18]. Our GRAPHMEMGUARD is a lightweight realisation of provenance-aware retrieval: edges carry their owning session and sensitivity label, and the policy decides admission *before* the query result enters the response context. We evaluate it both as a strict isolation baseline (b=0) and as a budgeted exposure mechanism (b>0).

III. SYSTEM AND THREAT MODEL

A. Dynamic KG Memory

We model a graph memory as $\mathcal{G}_t = (V_t, E_t, \pi)$ at logical time t, where every edge $e \in E_t$ carries provenance $\pi(e) = (\textit{owner}, \textit{user}, \textit{turn}, \textit{sensitivity}, \textit{ts})$. An edge is added when a session s writes a relation extracted from a turn *turn*. The service exposes a retrieval primitive $\textsc{Retrieve}(q, s_r, k) \to R$ that returns the k-hop expansion around query q from the perspective of requester session s_r. Visibility is governed by a guard Γ such that each edge is admitted only if $\Gamma(e, s_r) = \text{True}$.

B. Adjacency

For privacy analysis we adopt *edge-event adjacency*: two histories $\mathcal{G}, \mathcal{G}'$ are neighbours if they differ in exactly one provenance-tagged edge e. This matches the granularity at which our write-time admission mechanism operates and admits a future composition argument over t writes.

C. Adversaries

We consider three adversaries:

1) **Black-box requester** ($\mathcal{A}_b$). Issues queries and observes only final responses.
2) **Gray-box requester** ($\mathcal{A}_g$). Additionally observes the retrieved subgraph (a common setting for transparency or developer modes).
3) **Honest-but-curious operator** ($\mathcal{A}_o$). Inspects metadata for ground-truth evaluation; never used as an attacker against a deployed system, only to audit.

Our experiments instantiate $\mathcal{A}_g$ because it is the strictly stronger of the two practical adversaries and gives an upper bound on cross-session leakage measurable from the retrieval API.

D. Sensitive Objects

The sensitive objects are: (i) edge existence $\mathbf{1}[e \in E_t]$, (ii) edge ownership $\textit{owner}(e) = s_v$, (iii) the local ego-graph of a victim session, and (iv) the temporal sequence of writes producing a sensitive neighbourhood.

E. Out-of-Scope

We do *not* model attacks against the underlying storage layer, side channels of the retrieval service, or model-internal memorisation. We assume responses are produced by a downstream component and focus on the graph retrieval boundary.

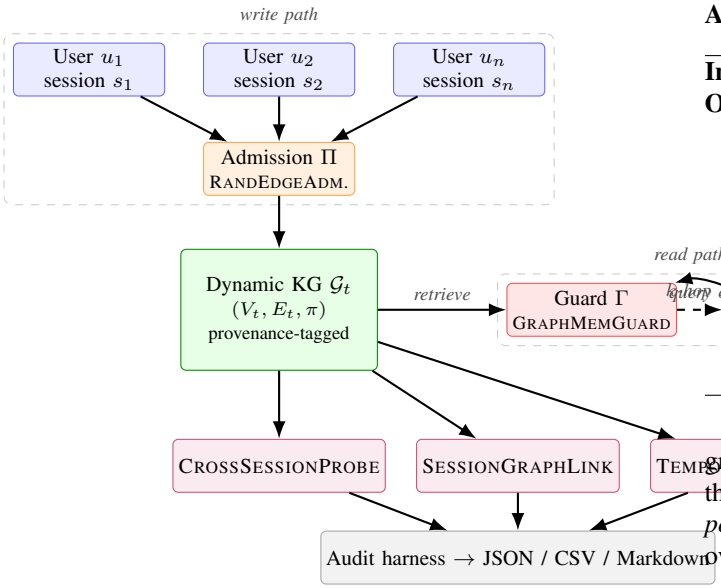

Fig. 1. System architecture of GraphMemShield. Write-time admission (Π) and read-time guard (Γ) bracket the dynamic KG; the audit harness exposes attack and metric modules.

IV. THE GRAPHMEMSHIELD FRAMEWORK

A. Overview

Figure 1 shows the framework. Sessions write into the dynamic KG via an admission policy Π. Retrieval consults the KG through a guard Γ. Three pluggable attack modules consume the audit interface, and an evaluation harness aggregates JSON, CSV, and Markdown reports.

B. Attack 1: CROSSSESSIONPROBE

Given an attacker session s_a and a victim session s_v, the probe issues a budgeted set of queries Q and counts retrieved edges whose owner is s_v. We report (i) *leaked edge count* (unique victim-owned edges over Q), (ii) *leakage events* (occurrences across queries), (iii) the *leakage rate* $|L|/|U|$ where L are leaked edges and U are unique retrieved edges, (iv) the *event leakage rate*, and (v) the *per-query leak rate*.

C. Attack 2: SESSIONGRAPHLINK

Each session is mapped to a multiset of features: relation counts, relation bigrams, sensitivity bigrams, relation co-occurrence pairs, and a degree histogram. The attacker ranks candidate sessions by cosine similarity to a query session. Optionally, semantic labels (entity tokens) are added, modelling a weak pseudonymisation regime. Reported metrics are top-k accuracy and Mean Reciprocal Rank (MRR).

D. Attack 3: TEMPORALPATHINFER

Given a query and a k-hop expansion, the attacker orders the returned edges by their observable timestamp metadata. We report (i) *prefix ordering accuracy* for the leading $|P|$ edges, (ii) *pairwise ordering accuracy* restricted to comparable ground-truth pairs, and (iii) edge precision/recall/F1 against the ground-truth path. This is deliberately a *provenance exposure* baseline; stronger attackers that perform beam search over hidden timestamps are future work.

Algorithm 1. GUARDEDRETRIEVE

Input: query q, requester s_r, hops k, guard Γ
Output: retrieval result R

1) $S \leftarrow \{e \in E : \text{match}(e, q) \wedge \Gamma(e, s_r)\}$
2) Initialise BFS queue with endpoints of S, depth 0
3) While queue non-empty and depth $< k$:
   - Pop (v, d)
   - For each $e \in \text{adj}(v)$ s.t. $\Gamma(e, s_r)$: $S \leftarrow S \cup \{e\}$; enqueue neighbour at $d{+}1$
4) For $e \in S$: record exposure Γ.RECORD(e, s_r)
5) **return** $R = (q, s_r, S, \text{nodes}(S))$

E. Defense 1: GRAPHMEMGUARD

GRAPHMEMGUARD is parameterised by a triple $(\alpha, b, \mathcal{B})$ where $\alpha \in \{0, 1\}$ enables cross-session sharing, $b \in \mathbb{Z}_{\geq 0}$ caps the unique cross-session edges admitted per requester–owner pair, and $\mathcal{B}$ is the set of blocked sensitivity labels. The predicate is

$$\Gamma(e, s_r) = \begin{cases} \text{True} & \textit{owner}(e) = s_r \\ \text{False} & \textit{sens}(e) \in \mathcal{B} \\ \text{False} & \alpha = 0 \\ \mathbf{1}[c(s_r, \textit{owner}(e)) < b] & \text{otherwise} \end{cases} \tag{1}$$

where $c(\cdot, \cdot)$ tracks unique cross-session admissions per pair. The strict configuration $\alpha = 0$, $b = 0$ realises full provenance/session isolation and yields a hard upper bound on the defense.

F. Defense 2: RANDOMIZEDEDGEADMISSION

This write-time module accepts a sensitive edge with probability $p_\varepsilon = \sigma(\varepsilon)$, where σ is the sigmoid. The decision is seeded by a SHA-256 keyed hash of the edge ID, which keeps experiments deterministic but is not a substitute for a noise draw from a formal mechanism. We discuss in Section VII the additional adjacency and composition work required to lift this proxy to a full ε-edge-DP guarantee.

G. Algorithmic Summary

Algorithm IV-G states the guarded retrieval procedure that powers all reported numbers.

V. IMPLEMENTATION

A. Open-source Package

The framework is published as the Python package `graphmemshield`, organised into `core/` (graph and types), `attacks/`, `defenses/`, `datasets/`, `evaluation/`, and `adapters/`. The package is installed in editable mode for reproducible experiments and is exercised by 22 unit tests covering ingestion, retrieval, attacks, defenses, and metrics.

B. Datasets

Two datasets back the reported numbers:

Synthetic multi-session graph. A deterministic graph with two "Alice" sessions (medical edges) and one "Bob" session (financial edges). The repeated (*visited*, *diagnosed*) motif across Alice's sessions is designed to expose both cross-session retrieval contamination and structural linkage.

Docker-backed sample dialogues. A 12-record JSONL corpus across 6 users, 12 sessions, and four domains (*health, finance, work, location*). Records are seeded into a mock PrivacyGuard MongoDB store and re-fetched through a Cloud REST API before being mapped into 36 nodes and 48 edges. Sensitivity labels are propagated through the adapter so that GRAPHMEMGUARD can apply the policy from Equation (1).

C. Reproducibility

Each result in this paper is reproduced by:

```
python examples/run_experiments.py
python examples/run_privacyguard_batch_experiment.py
python examples/generate_aggregate_report.py
```

which write `output/synthetic_experiments.{json,csv,md}`, `output/privacyguard_batch_results.{json,csv,md}` and the 62-row `output/aggregate_results.{json,csv,md}`.

TABLE I
CROSS-SESSION LEAKAGE (RQ1, RQ2). STRICT GUARD USES $(\alpha{=}0, b{=}0, \mathcal{B})$.

System	Dataset	Cond.	Leak.	Red.
Synthetic sim.	multisession	baseline	2	–
Synthetic sim.	multisession	strict_guard	0	1.00
PG-Docker	seed	baseline	5	–
PG-Docker	seed	strict_guard	0	1.00
PG-Docker	sample	baseline	48	–
PG-Docker	sample	strict_guard	0	1.00

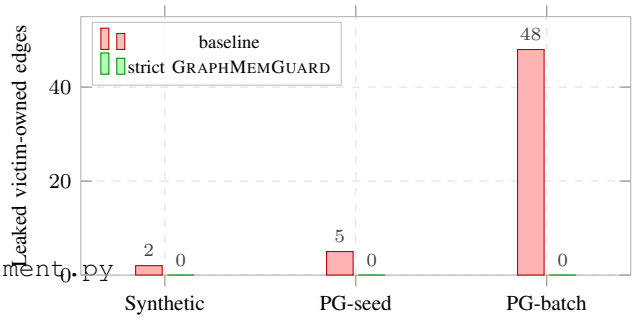

Fig. 2. Leaked victim-owned edges before and after applying GRAPHMEMGUARD (strict). Strict provenance/session isolation eliminates measured leakage on both datasets.

VI. EXPERIMENTAL EVALUATION

A. Research Questions

RQ1. How much victim-owned graph memory does a baseline graph-memory service expose to an unrelated session?
RQ2. Does provenance/session isolation eliminate the exposure, and at what budget level can controlled cross-session sharing be allowed without leaking sensitive labels?
RQ3. How effectively can structural fingerprints re-identify anonymised sessions, and what is the gain from semantic labels?
RQ4. What does observable provenance metadata reveal about write-order and sensitive paths?
RQ5. How does a write-time edge-admission proxy trade sensitive-edge retention for downstream leakage as ε varies?

B. Experimental Setup

We instantiate $\mathcal{A}_g$ with a fixed query bank derived from each domain's seed entities (e.g., *clinic*, *arrhythmia*, *diagnosed*, *gym*, *visited*). All retrievals use $k{=}1$. Strict guard runs use the policy $(\alpha{=}0, b{=}0, \mathcal{B}{=}\{\textit{secret}, \textit{medical}, \textit{financial}\})$. Budgeted runs vary $b \in \{0, 1, 2\}$. Edge-admission runs sweep $\varepsilon \in \{0.1, 1.0, 3.0\}$.

C. RQ1 & RQ2: Cross-Session Exposure

Table I reports the headline numbers. On the synthetic graph the baseline retrieves 2 unique victim-owned edges (6 leakage events across 3 probe queries, leakage rate 0.5); on the Docker-backed batch graph the baseline exposes *all* 48 victim-owned edges across 48 queries. Strict isolation reduces both to zero, yielding a leakage reduction of 1.0 in both regimes. The visualisation in Figure 2 makes the gap explicit.

1) Query-Budget Curve: Figure 3 sweeps the per-session query budget on the Docker-backed sample dataset. A single entity-anchored query per session (budget $b{=}1$) already exposes the full set of 48 victim-owned edges; increasing the budget multiplies the *event* count (re-retrievals across queries) without raising the unique-edge count. This is a useful auditing signal: it shows that adversarial *coverage* is determined by the schema of the exposed entity surface, not by the number of probes, and that single-shot defenses are sufficient to bound exposure under our adjacency definition.

D. RQ3: Session Linkage

Table II shows that on the Docker-backed sample dataset SESSIONGRAPHLINK achieves Top-1 accuracy of 0.33 from *structure only* (relation, sensitivity, motif and degree features), with all correct matches falling within the Top-3 (1.00 Top-3 accuracy). Adding semantic labels raises MRR from 0.61 to 0.67 but does not change Top-1 in this small regime. Figure 4 visualises the comparison.

E. RQ4: Temporal Path Exposure

On the Docker-backed sample dataset TEMPORALPATHINFER achieves prefix ordering accuracy 0.50, pairwise ordering accuracy 1.00, and edge precision/recall/F1 of $0.49/1.00/0.66$ across 12 sessions. This empirically confirms that exposing `created_at` metadata in the retrieval payload is sufficient to recover the relative write-order of every reachable edge; the recall of 1.00 follows directly from the strong adjacency between query terms and seed entities.

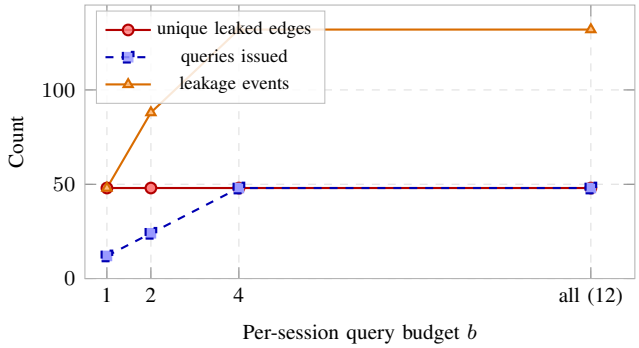

Fig. 3. Query-budget curve on the Docker-backed sample dataset. Unique leaked edges saturate at b=1; leakage events scale linearly with retrieval count.

TABLE II
SESSION-LINKING RESULTS ON DOCKER-BACKED SAMPLE DATASET.

Feature condition	Top-1	Top-3	MRR
structure-only	0.333	1.000	0.611
structure + semantic	0.333	1.000	0.667

F. RQ5: Edge-Admission Sweep

Figure 5 reports the synthetic edge-admission sweep for $\varepsilon \in \{0.1, 1.0, 3.0\}$. The keep-probability rises from 0.525 to 0.953 across the sweep, while the number of admitted sensitive edges climbs from 3 to 5 and the resulting victim leakage rises from 1 to 2 unique edges. We emphasise that the seeded-hash mechanism is a controlled proxy: the curve is *indicative of* the privacy–utility surface of a future provable mechanism rather than a guarantee of edge differential privacy.

G. Bounded-Sharing Mode

For completeness, the budgeted policy (α=1, $\mathcal{B}$={*medical*}) is exercised on the synthetic graph against the non-medical Bob session. Increasing the per-pair budget b from 0 to 2 admits 0, 1, and 2 cross-session edges respectively, with medical edges remaining blocked. This illustrates that GRAPHMEMGUARD can interpolate between strict isolation and bounded sharing, providing operators with a direct lever for the privacy–utility tradeoff at retrieval time.

VII. DISCUSSION, SCOPE, AND ETHICS

A. What the Numbers Mean

The headline result—48 leaked edges → 0 leaked edges—is a strict-isolation baseline. It demonstrates that the audit pipeline correctly observes the dynamic KG memory and that the provenance guard severs the cross-session boundary. The interesting operational regime is the budgeted middle, where allowing b=1–2 cross-session edges per session pair preserves utility for collaborative queries while still excluding sensitive labels. The query-budget curve confirms the auditor's intuition that, under the baseline policy, exposure saturates at the smallest non-trivial adversarial budget, so single-shot retrieval-time defenses are appropriate.

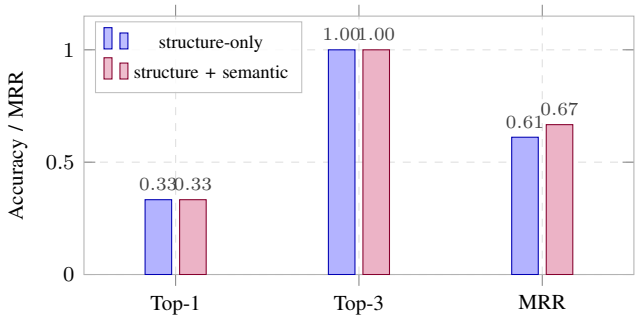

Fig. 4. SESSIONGRAPHLINK accuracy. Adding semantic tokens improves MRR but not Top-1 in this regime.

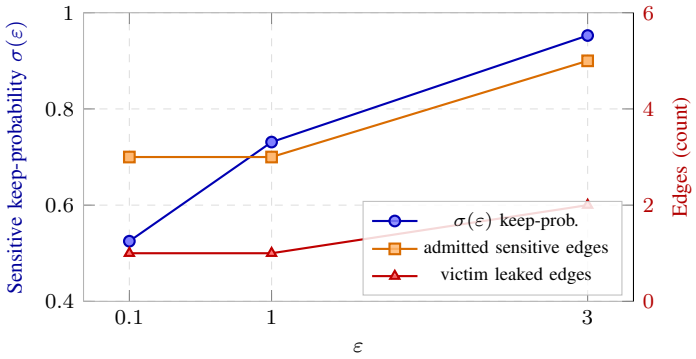

Fig. 5. RANDOMIZEDEDGEADMISSION sweep on the synthetic multisession graph. Keep-probability follows $\sigma(\varepsilon)$; admitted sensitive edges and downstream leakage rise monotonically.

B. Scope and Limitations

The current artefact is intentionally framed as a benchmark and defensive baseline, not as a fully formal mechanism. We emphasise: (i) the dataset is small and de-identified; large-scale validation on PersonaChat, MultiWOZ, or controlled health/finance corpora is queued as future work; (ii) the Docker integration exercises the read-path through the Cloud API but not the full `validate-and-prove` pipeline of the underlying PrivacyGuard mock; (iii) SESSIONGRAPHLINK is heuristic and does not yet include Weisfeiler–Lehman or graph-edit-distance baselines; (iv) TEMPORALPATHINFER measures provenance exposure rather than hidden-order inference, and a beam-search variant is needed for the no-timestamp regime; (v) RANDOMIZEDEDGEADMISSION is a seeded proxy and lacks the adjacency, accounting, and composition proofs required for an ε-edge-DP claim. We stress these limits because the strict-guard reduction of 1.0 would otherwise overstate the framework's contribution.

C. Ethical Considerations

All datasets used in this paper are synthetic or de-identified; no real users or production systems were probed. The framework is released for defensive auditing of graph-backed application memories. Operators integrating GraphMemShield should publish their chosen $(\alpha, b, \mathcal{B})$ policy and document the corresponding upper bound on cross-session exposure.

VIII. Conclusion

We presented GraphMemShield, a reproducible auditing and mitigation framework for cross-session privacy leakage in dynamic KG memories of graph-backed applications. The framework formalises edge-event adjacency, provides three baseline attacks targeting cross-session exposure, structural linkage, and provenance ordering, and ships two defenses spanning retrieval-time isolation and write-time admission. On the Docker-backed batched workload, strict provenance/session isolation removes *all* 48 victim-owned cross-session edges, while a query-budget sweep clarifies that exposure is determined by schema coverage rather than probe count. We release the open-source artefact, document the gap between the current proxy and a fully provable mechanism, and outline the dataset, accounting, and adversary extensions that constitute the next milestone.

References

[1] N. Carlini *et al.*, "Extracting training data from large language models," in *Proc. USENIX Security*, 2021.

[2] F. Mireshghallah, A. Goyal, A. Uniyal, T. Berg-Kirkpatrick, and R. Shokri, "Memorisation in fine-tuned dialogue models," in *Proc. EMNLP*, 2023.

[3] W. Zeng, K. Liu, R. Lin, and B. Hooi, "The good and the bad: Exploring privacy issues in retrieval-augmented generation (RAG)," *arXiv:2402.16893*, 2024.

[4] Y. Qi *et al.*, "Follow my instruction and spill the beans: Scalable data extraction from retrieval-augmented generation systems," *arXiv:2402.17840*, 2024.

[5] J. Wang, Z. Lin, T. Xu, and Q. Liu, "Exposing privacy risks in graph retrieval-augmented generation," *arXiv:2503.05923*, 2025.

[6] X. He, J. Jia, M. Backes, N. Z. Gong, and Y. Zhang, "Stealing links from graph neural networks," in *Proc. USENIX Security*, 2021.

[7] B. Wu, X. Yang, S. Pan, and X. Yuan, "Model inversion attacks against graph neural networks," *IEEE Trans. Knowledge and Data Engineering*, vol. 35, 2022.

[8] I. E. Olatunji *et al.*, "Membership inference attacks on graph neural networks," in *Proc. ACSAC*, 2023.

[9] V. Duddu, A. Boutet, and V. Shejwalkar, "Quantifying privacy leakage in graph embedding," in *Proc. MobiQuitous*, 2020.

[10] M. Hay, C. Li, G. Miklau, and D. Jensen, "Accurate estimation of the degree distribution of private networks," in *Proc. IEEE ICDM*, 2009.

[11] Z. Jorgensen, T. Yu, and G. Cormode, "Publishing attributed social graphs with formal privacy guarantees," in *Proc. ACM SIGMOD*, 2016.

[12] V. Karwa, S. Raskhodnikova, A. Smith, and G. Yaroslavtsev, "Private analysis of graph structure," *ACM Trans. Database Systems*, vol. 39, 2014.

[13] A. Daigavane *et al.*, "Node-level differentially private graph neural networks," in *Proc. ICLR Workshop*, 2021.

[14] S. Sajadmanesh and D. Gatica-Perez, "GAP: Differentially private graph neural networks with aggregation perturbation," in *Proc. USENIX Security*, 2023.

[15] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, "Membership inference attacks against machine learning models," in *Proc. IEEE S&P*, 2017.

[16] A. Salem *et al.*, "ML-Leaks: Model and data independent membership inference attacks and defenses on machine learning models," in *Proc. NDSS*, 2019.

[17] M. Backes *et al.*, "Walk2friends: Inferring social links from mobility profiles," in *Proc. ACM CCS*, 2017.

[18] T. Pasquier *et al.*, "Practical whole-system provenance capture," in *Proc. ACM SoCC*, 2017.